L2L ANALYTICS CHATBOT - PROJECT STATUS REPORT

Comprehensive Architecture, Implemented Features & Technical Milestones

DATE: October 2026 | **DOMAIN:** Leading2Lean Manufacturing Operations | **STATUS:** **Operational & Production-Ready**

WORKSPACE: C:\AI_dev\l2l-analytics-chatbot | **STACK:** FastAPI, HTMX, LangGraph, DuckDB, Plotly

1. EXECUTIVE SUMMARY & OBJECTIVES

The L2L Analytics Chatbot is a full-stack, enterprise manufacturing intelligence assistant designed to query, analyze, and visualize Leading2Lean (L2L) dispatch and maintenance metrics in real time. It enables plant supervisors, reliability engineers, and process leaders to interact with plant operations data using natural language, receiving both structured markdown tables and responsive, interactive Plotly visualization charts.

2. ARCHITECTURAL HIGHLIGHTS & TWO-TIER DATA STRATEGY

- High-Performance Local OLAP Cache (BigQuery -> DuckDB):

Direct LLM querying of cloud data warehouses like Google BigQuery introduces latency, cost, and quota limits. The solution implements an automated pipeline in src/cache.py syncing cons_kamos_l2l.f_l2l_dispatches into a local embedded DuckDB database (/app/data/l2l_analytics.duckdb) with a 10s query timeout & 2GB memory cap.

- Cyclic LangGraph ReAct Workflow:

Employs a StateGraph in src/agent.py looping between an LLM agent node and a tool execution node. The agent decides whether to execute DuckDB analytical SQL, generate Plotly charts, or directly answer queries.

- Persistent Multi-Turn Memory via PostgresSaver:

Integrated with PostgreSQL 16 (l2l_chat_memory) via a connection pool. Every conversation retains an isolated thread_id allowing full session restoration, historical chart re-rendering, and seamless context switching.

3. TECHNOLOGY STACK SPECIFICATIONS

LAYER	COMPONENTS & VERSIONS	ROLE & CAPABILITY
Backend API	FastAPI 0.110.0 + Uvicorn 0.28.0	Async REST endpoints, HTMX partials, session handlers
Hypermedia UI	HTMX 1.9.10 + Tailwind CSS + Jinja2	Server-Side Rendered (SSR) interactive single-page app
Visualizations	Plotly.py 5.20.0 + Plotly.js 2.30.0	Responsive dark-themed bar, line, pie, and scatter charts
Agent Engine	LangChain 0.2 + LangGraph 0.1	Cyclic ReAct execution graph, tool invocation
LLM Providers	OpenAI (gpt-4o) / Google Gemini 1.5	Configurable providers via .env with zero temperature
OLAP Database	DuckDB 0.10.1 + BigQuery 3.19.0	Local high-performance columnar dispatch cache
Memory Saver	PostgreSQL 16 Alpine + psycopg 3	PostgresSaver checkpointer for long-term multi-session memory
Protocols / DevOps	FastMCP 0.1.0 + Docker Compose	Model Context Protocol server, container orchestration

4. IMPLEMENTED SUBSYSTEMS & CAPABILITIES

4.1 Text-to-SQL Analytics & Safe Execution Engine (src/tools.py, src/cache.py)

- query_l2l_database tool translates user questions into DuckDB SQL and outputs formatted Markdown tables.
- Read-only database access enforcing strict statement limits: SET statement_timeout = '10s', max_memory = '2GB'.
- Handles empty records gracefully and returns descriptive SQL errors that allow the LLM agent to self-correct.
- Cache sync endpoint (POST /sync-cache) enables on-demand ingestion from Google BigQuery.

4.2 Dynamic Plotly Chart Generation (src/tools.py)

- generate_l2l_chart tool accepts SQL query, chart_type ('bar', 'line', 'pie', 'scatter'), axes columns, and title.
- Constructs full Plotly figures using 'plotly_dark' theme matching the sleek dark web UI.
- Wraps serialized JSON in '''plotly_json code fences, ensuring uncorrupted transmission through LLM context.
- Automatically scrubs redundant LLM boilerplate, keeping the interface clean and visual-focused.

4.3 Hypermedia Reactive Frontend (FastAPI + HTMX + Tailwind + Plotly)

- Eliminates complex React/Node.js builds by serving fast, reactive Jinja2 partials via HTMX.
- Form submission via hx-post='/chat' streams turns smoothly into the message pane using hx-swap='beforeend'.
- Uses Out-Of-Band (OOB) swaps to update active thread IDs and chat titles in real time without page reload.
- Emits HX-Trigger: refreshSidebar so newly named conversation sessions appear instantly in the sidebar.
- Client-side Plotly script renders the interactive chart canvas as soon as the partial arrives in the DOM.

4.4 Multi-Session Thread Management & History Replay (src/utils/helpers.py)

- Automated semantic thread titles: DDMonYY-HH:MM:SS-<Slug> (e.g., 26Sep26-13:33:00-Site-240-Dispatches).
- Interactive sidebar with active session highlighting and a 'New Chat' launcher.
- extract_history_turns reconstructs prior turns and re-renders historical Plotly charts with full interactivity.
- GET /chat/history/{thread_id} reloads prior discussions smoothly with zero loss of context.

5. MANUFACTURING DOMAIN KNOWLEDGE & ENTITY RESOLUTION

The dynamic prompt generator (src/prompts/agent_prompt.py) provides calendar grounding and plant domain rules:

FACILITY / SITE MAPPING RULES:

- Site 240 <---> Stargard (e.g., 'Stargard dispatches' automatically maps to: WHERE site = 240)
- Site 260 <---> Shanghai (e.g., 'Shanghai downtime' automatically maps to: WHERE site = 260)
- Site 276 <---> Ottawa (e.g., 'Ottawa open dispatches' automatically maps to: WHERE site = 276)

CORE DATABASE SCHEMA ATTRIBUTES (Table: f_l2l_dispatches):

- dispatchnumber (BIGINT): Unique identifier
- dispatchtype (VARCHAR): Category of dispatch
- downtime (BIGINT): Total duration of downtime
- Open (BOOLEAN): Status (true = open, false = closed)
- dispatch_reported (TIMESTAMPTZ): Creation timestamp
- lineabbreviation / linecode: Production line
- reasoncode (VARCHAR): Downtime reason code
- site (BIGINT): Facility ID (240, 260, 276)
- machinecode (VARCHAR): Specific machine code
- sigdowntime (BIGINT): Significant downtime metric
- tradecode (VARCHAR): Maintenance trade required
- completed (TIMESTAMPTZ): Completion timestamp
- areacode (VARCHAR): Area within plant
- current_dispatchstatus_description (VARCHAR)

Temporal Logic: Real-time awareness calculates current date/year, translating queries like 'last 3 months' into DuckDB SQL expressions: WHERE dispatch_reported >= CURRENT_DATE - INTERVAL '3 months'.

6. PROTOCOLS, SAFETY GUARDRAILS & CONTAINER DEPLOYMENT

6.1 Model Context Protocol Integration (src/mcp_server.py)

- Implemented with FastMCP (0.1.0) to expose DuckDB querying tools over the open MCP standard.
- Enables external MCP-compatible AI assistants (Claude Desktop, Cursor, IDEs) to interact directly with the manufacturing data cache without modifying backend application code.

6.2 Input Safety & Guardrails (config/config.yml)

- Configured NeMo Guardrails (>=0.9.0) with input rails protecting against prompt injection and malicious inputs.
- Establishes boundary enforcement to prevent unauthorized table modifications or SQL tampering.

6.3 Multi-Container Docker Architecture (docker-compose.yml, Dockerfile)

- Web Container: Python 3.11-slim with compiled C-extensions, live code reload, and mounted DuckDB volume.
- PostgreSQL 16 Alpine: Healthchecked service with persistent volume (postgres_data) for conversational state.
- GCP Authentication: Mounts host Google Application Default Credentials for seamless BigQuery data ingestion.

7. WORKSPACE FILE INVENTORY & ARTIFACTS

FILE PATH	FUNCTIONALITY & STATUS
main.py	FastAPI web entrypoint, HTMX chat routes, session history endpoints, test harnesses
docker-compose.yml	Orchestrates FastAPI app, DuckDB volume, host GCP auth, and PostgreSQL 16
Dockerfile	Python 3.11-slim container with build essentials and pinned dependencies
requirements.txt	Pinned dependencies: FastAPI, DuckDB, LangGraph, Plotly, FastMCP, psycopg
src/agent.py	LangGraph StateGraph workflow, tool execution node, PostgresSaver checkpointer
src/cache.py	BigQuery data sync pipeline, local DuckDB cache materialization, safe query executor
src/tools.py	LangChain tools: query_l2l_database & generate_l2l_chart (Plotly JSON)
src/prompts/agent_prompt.py	Dynamic system prompt with calendar context and plant site resolution logic
src/utils/helpers.py	Semantic thread name generator, Postgres session query, history turn parser
src/mcp_server.py	FastMCP server exposing DuckDB query tools to external MCP AI assistants
templates/ & static/	Jinja2 templates: base.html, sidebar.html, chat_area.html, HTMX response partial

8. SUMMARY & OPERATIONAL READINESS

The L2L Analytics Chatbot has accomplished all core milestones for an end-to-end manufacturing intelligence platform. The system achieves sub-second response times through local DuckDB OLAP caching, retains multi-turn sessions across restarts via PostgreSQL checkpoints, autonomously generates production-grade Plotly charts, and provides an agile, no-build web interface using HTMX and Tailwind CSS.

The platform is fully containerized and verified for production or staging deployment.